Mohamed bin Zayed University of Artificial Intelligence

Coding Patterns

Week 5 — Depth over breadth: patterns as decision tools

Strategy | Observer | Adapter | Repository | Command | State

What you'll learn

- Why patterns exist (recurring pain → reusable structure)
- How to choose a pattern using practical "selection lenses"
- How to adopt patterns incrementally without a rewrite

Depth over breadth

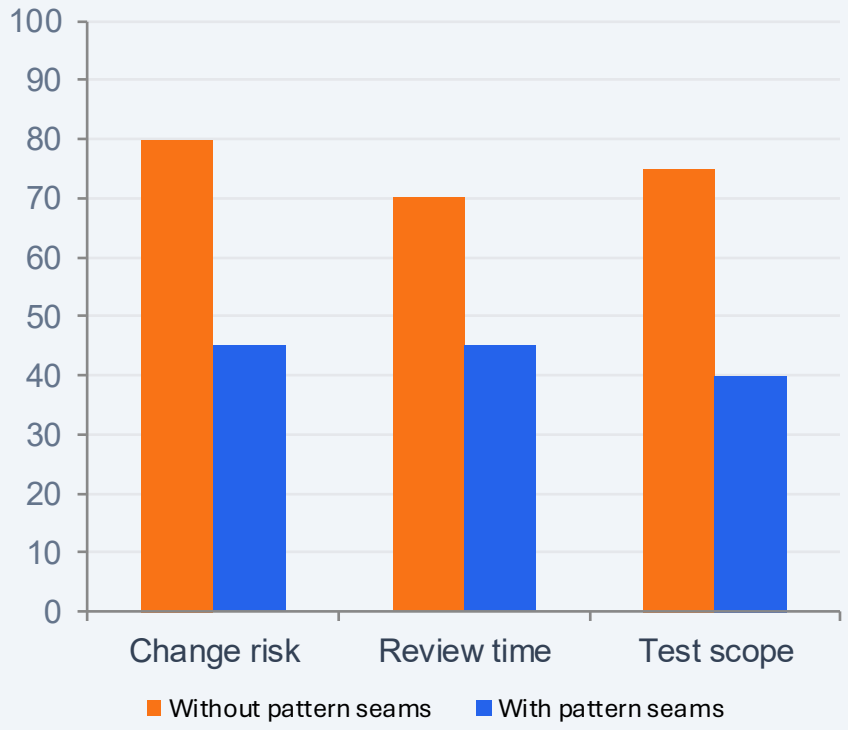
Patterns reduce risk — mastering the “why” builds reusable judgment.

Mohamed bin Zayed University of Artificial Intelligence

Why fewer patterns?

- Patterns are tools: pick the smallest one that removes today’s pain.
- Consistency speeds onboarding, reviews, and refactors.
- Real examples reveal where teams lose time and reliability.

Illustrative impact (not real data)



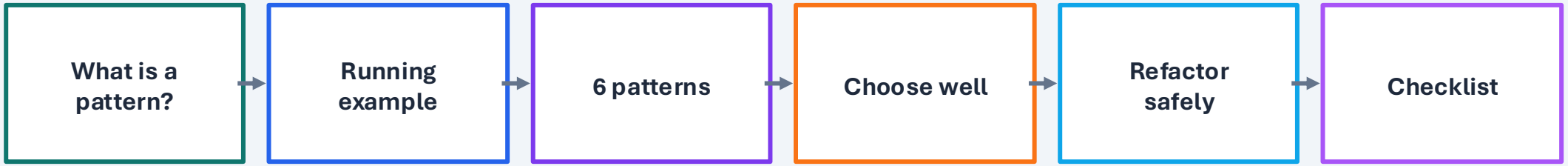
Goal: safe change speed

Agenda

A practical walk-through: problem → pattern → tradeoffs → adoption.

Mohamed bin Zayed University of Artificial Intelligence

Lesson flow



- We'll compare similar patterns so you can choose based on requirements.
- You'll see incremental refactoring paths (no rewrites).

Anatomy of a pattern

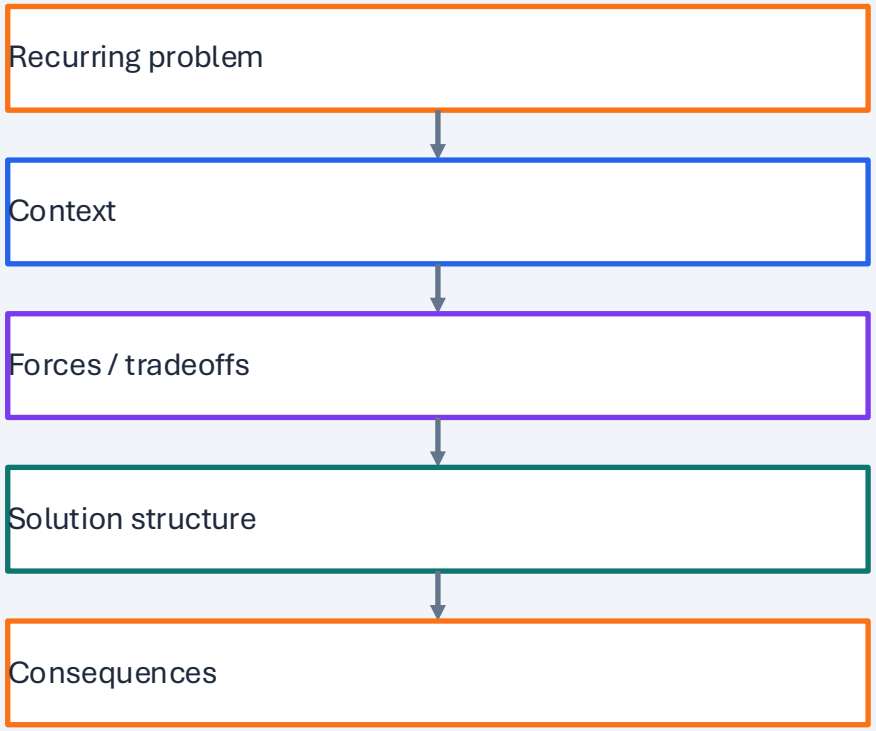
Patterns aren't templates — they are tradeoff-aware solution structures.

Mohamed bin Zayed University of Artificial Intelligence

A pattern includes...

- Problem that keeps reappearing
- Context where it's safe to apply
- Forces & tradeoffs (why it's not one-size-fits-all)
- Solution structure you can adapt
- Consequences (the cost you're paying)

Cargo-cult prevention



If you can't name the problem + tradeoff, you're copying, not designing.

Why patterns exist

Recurring pain → predictable structure.

Mohamed bin Zayed University of Artificial Intelligence

Example pain: enrollment rules grow every semester

- Rules pile into long if/else chains across files.
- One tweak breaks an unrelated rule (hidden coupling).
- Tests become brittle when logic mixes with database calls.
- Releases slow down because behavior is hard to explain.

From chaos → seams

Before

EnrollmentService
+ if/else rules
+ SQL calls

Hard to test

After

Pricing (Strategy)

↓

Data access (Repository)

↓

Notifications (Observer)

Small units → fast tests

Pattern selection lenses

Use these questions to justify a pattern (instead of copying one).

Mohamed bin Zayed University of Artificial Intelligence

Six lenses

Change frequency

What changes often (rules, vendors, states)?

Ownership boundaries

Who owns each part? How many teams coordinate?

Testability

Can we test logic without DB / network?

Runtime constraints

Is indirection acceptable in hot paths?

Failure modes

Can one failure block the whole action?

Observability

Can we trace behavior with logs + IDs?

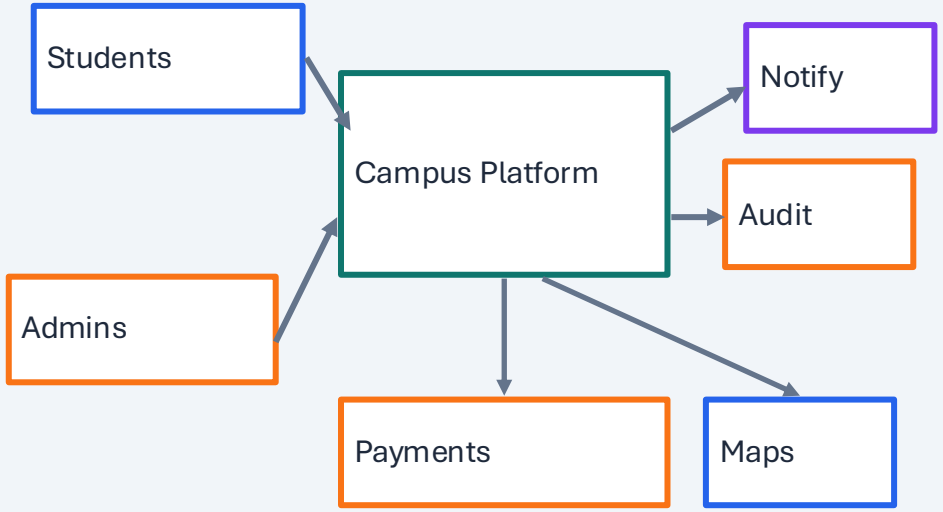
Running example: Campus Services Platform

Enrollment, shuttles, and events — with real integrations & audit needs.

Scope (student-sized, real-world pain)

- Different student categories → different pricing & eligibility rules.
- Notifications when seats open or routes change.
- Third-party payments and map providers.
- Admins must audit and undo enrollment changes.

System context



This single example generates all six pattern problems.

Strategy pattern — problem

Strategy

Pricing and eligibility rules multiply over time.

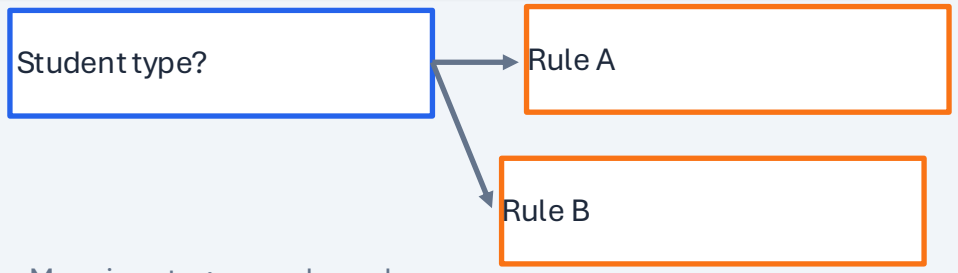
Symptoms

- One pricing function grows into nested if/else.
- Adding a scholarship means editing core logic in many places.
- Bugs appear because rule order matters.
- Testing is slow: you must exercise every branch combo.

Branch explosion

Typical outcome

```
function price(ctx) {
  if (ctx.student === 'staff') {
    if (ctx.event === 'sports') return ...
    else if (ctx.event === 'concert') return ...
  } else if (ctx.student === 'scholarship') {
    if (ctx.window === 'early') return ...
    else return ...
  }
  // ...more rules...
}
```



More inputs → more branches

Strategy pattern — solution

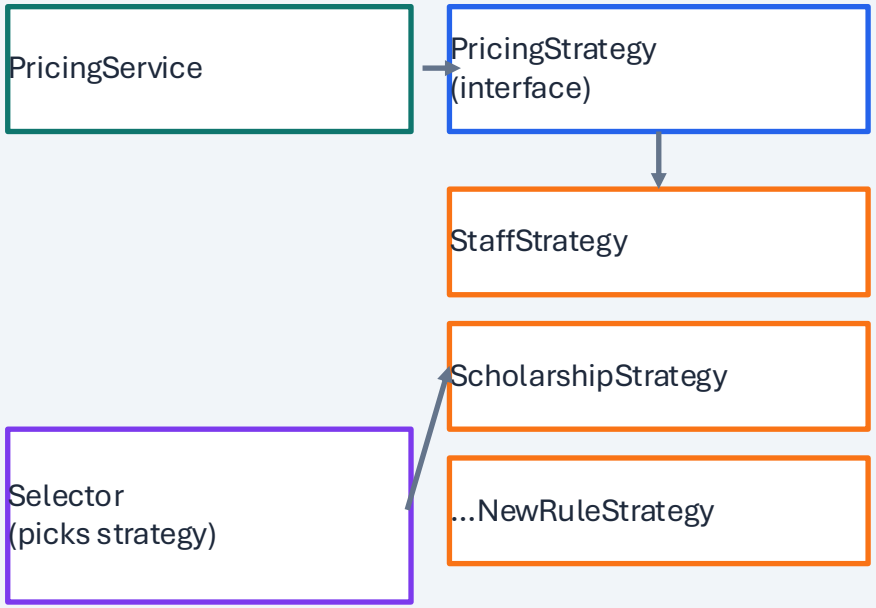
Strategy

Encapsulate each rule behind a shared interface.

How it helps

- Each rule becomes a strategy with a shared interface.
- Selection happens at runtime based on context.
- New rules = add a class, not edit the core.
- Strategies unit-test in isolation (lower regression risk).

Structure



Cost: more files. Benefit: safe change speed.

Strategy pattern — code example

Strategy

Each pricing rule is a separate class behind a shared interface.

```
// 1. Define the strategy interface
interface PricingStrategy {
  calculate(ctx: EnrollmentContext): Price;
}

// 2. Concrete strategies — one per rule
class StaffPricing implements PricingStrategy {
  calculate(ctx) { return ctx.basePrice * 0.5;  }  // 50% staff discount
}
class ScholarshipPricing implements PricingStrategy {
  calculate(ctx) { return 0;  }  // fully covered
}

// 3. Selector picks the right strategy at runtime
const strategies: Record<string, PricingStrategy> = {
  staff:       new StaffPricing(),
  scholarship: new ScholarshipPricing(),
};

// 4. Service uses the strategy — no if/else needed
function price(ctx: EnrollmentContext): Price {
  const strategy = strategies[ctx.studentType];
  return strategy.calculate(ctx);  // no branches!
}
```

Observer pattern — problem

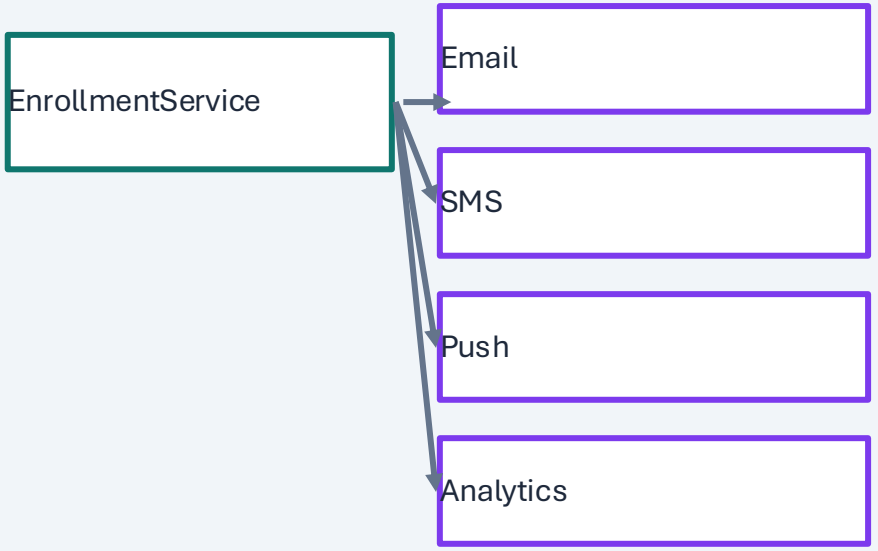
Observer

One event triggers many side effects — coupling grows fast.

Symptoms

- Enrollment service directly calls Email, SMS, Push modules.
- New channel → core service must change.
- One channel failure can block the entire action.
- Teams avoid adding features because integration points are fragile.

Tight coupling



Every arrow is a change + failure risk surface.

Observer pattern — solution

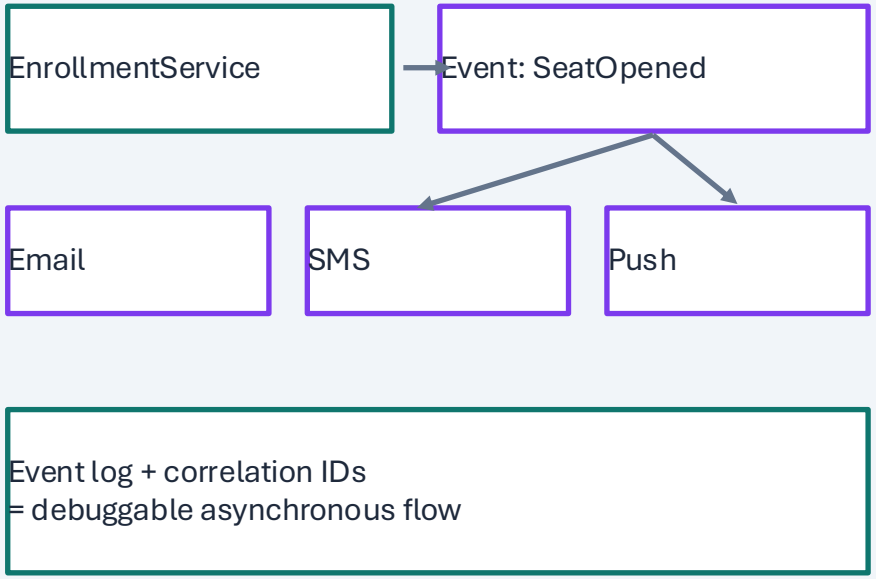
Observer

Publish an event; subscribers react independently.

How it helps

- Enrollment emits SeatOpened and moves on.
- Channels subscribe — add new ones without editing the core.
- Failures stay local; one bug won't stop others.
- Event logs become an audit-friendly trace.

Event-driven structure



Observer pattern — code example

Observer

Emit events; subscribers react independently without coupling.

```
// 1. Define an event type and a subscriber interface
type SeatEvent = { courseId: string; studentId: string };
interface SeatObserver { onSeatOpened(e: SeatEvent): void; }

// 2. Concrete subscribers — each handles one concern
class EmailNotifier implements SeatObserver {
  onSeatOpened(e) { sendEmail(e.studentId, "Seat available"); }
}
class AnalyticsTracker implements SeatObserver {
  onSeatOpened(e) { trackEvent("seat_opened", e); }
}

// 3. The service publishes events — knows nothing about subscribers
class EnrollmentService {
  private observers: SeatObserver[] = [];

  subscribe(obs: SeatObserver) { this.observers.push(obs); }

  dropStudent(courseId: string, studentId: string) {
    // ... remove student from course ...
    const event = { courseId, studentId };
    for (const obs of this.observers) obs.onSeatOpened(event);
  }
}

// 4. Wire it up — add new channels without touching the service
const svc = new EnrollmentService();
svc.subscribe(new EmailNotifier());
svc.subscribe(new AnalyticsTracker()); // add more without changing the service
```

Adapter pattern — problem

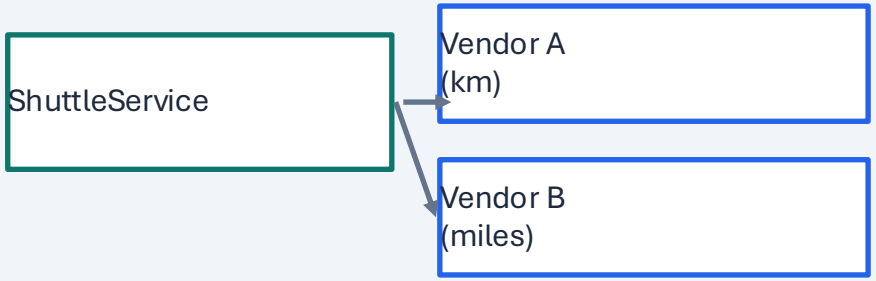
Adapter

Vendor APIs differ (units, shapes, errors) and change over time.

Symptoms

- Two map providers expose different route APIs and units.
- Conversion logic spreads across the codebase.
- Switching vendors breaks many call sites and tests.
- Error formats differ, making failures confusing to handle.

Incompatible interfaces



Unit conversions + error mapping leak into every call site.

Adapter pattern — solution

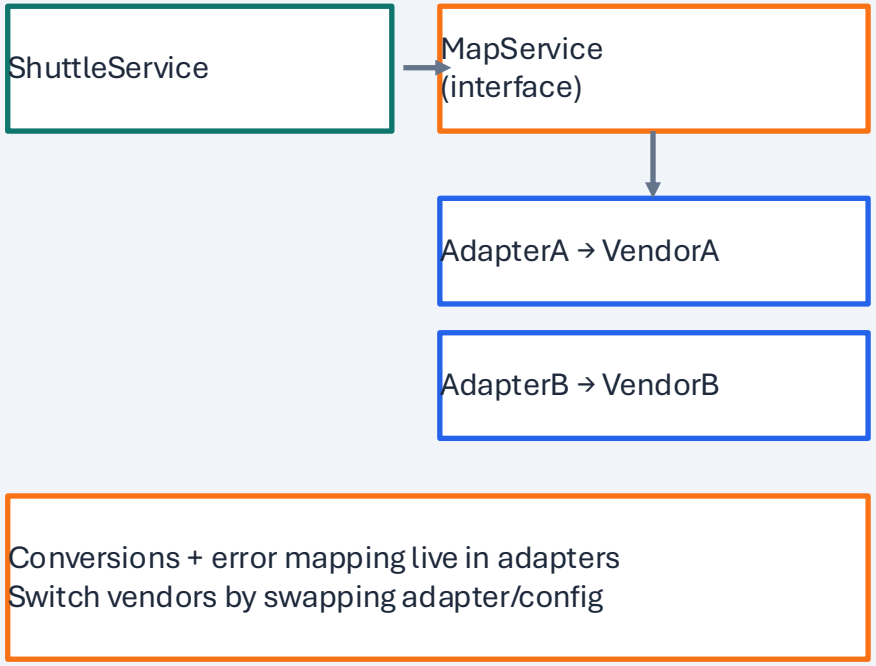
Adapter

Create one stable interface and translate each vendor into it.

How it helps

- Define MapService interface that matches platform needs.
- Adapters translate vendor APIs, units, and errors.
- Switch providers via configuration (not refactor).
- Tests use a fake adapter to avoid external dependencies.

Structure



Adapter pattern — code example

Adapter

One stable interface wraps each vendor; switch providers via config.

```
// 1. Your platform's stable interface
interface MapService {
  getRoute(from: LatLng, to: LatLng): Route;  // always returns km
}

// 2. Adapter for Vendor A (already uses km)
class VendorAAdapter implements MapService {
  getRoute(from, to) {
    const raw = vendorA.directions(from, to);  // vendor-specific call
    return { distanceKm: raw.dist, eta: raw.time };
  }
}

// 3. Adapter for Vendor B (converts miles → km)
class VendorBAdapter implements MapService {
  getRoute(from, to) {
    const raw = vendorB.fetchRoute(from, to);  // different API shape
    return { distanceKm: raw.miles * 1.609, eta: raw.duration }; // convert!
  }
}

// 4. ShuttleService depends only on MapService — never on vendors
class ShuttleService {
  constructor(private maps: MapService) {}  // inject any adapter
  estimateArrival(from, to) { return this.maps.getRoute(from, to).eta; }
}

// Switch vendors by changing one line:
const shuttle = new ShuttleService(new VendorBAdapter());  // swap here
```

Repository pattern — problem

Repository

Business rules leak SQL everywhere; tests get slow and fragile.

Symptoms

- Enrollment logic embeds SQL and query details throughout the codebase.
- Business logic mixes with persistence, making unit tests slow.
- Changing databases (SQLite → Postgres) becomes risky and messy.
- Duplicate queries creep in, causing bugs and inconsistencies.

SQL scattered

```
// inside service layer
db.query('SELECT ... WHERE open=1');
db.query('UPDATE seats SET ...');
// many files, many variants
```

Domain logic + persistence = hard seams

Result: refactors touch too many files.

Repository pattern — solution

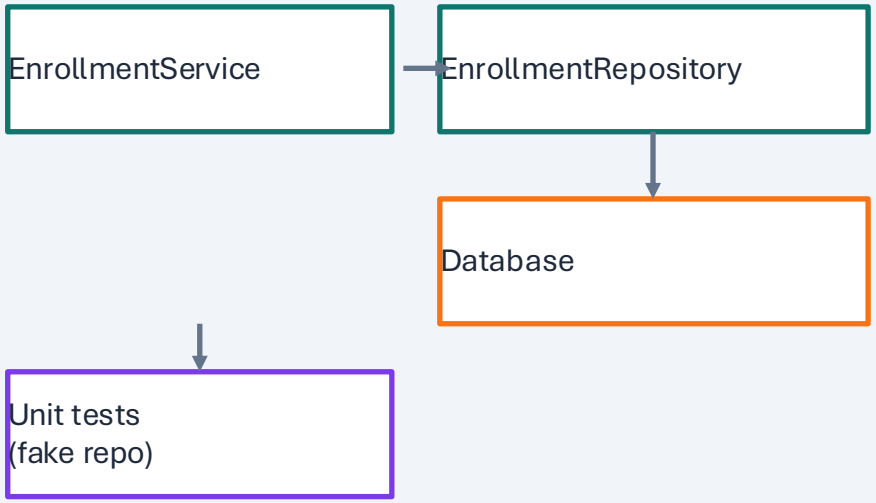
Repository

Expose domain-focused methods; keep SQL and optimization inside.

How it helps

- Repository exposes methods like findOpenSeats().
- Services call a clean API; logic stays readable.
- Mock repositories for fast, reliable unit tests.
- Database changes become localized.

Boundary



Cost: an extra abstraction. Benefit: stability and test seams.

Repository pattern — code example

Repository

Domain-focused methods hide SQL; services stay testable.

```
// 1. Repository interface — domain language, no SQL
interface EnrollmentRepo {
  findOpenSeats(courseId: string): Seat[];
  reserveSeat(courseId: string, studentId: string): void;
}

// 2. Real implementation — SQL lives only here
class PgEnrollmentRepo implements EnrollmentRepo {
  findOpenSeats(courseId) {
    return db.query(`SELECT * FROM seats WHERE course=$1 AND open=true`, [courseId]);
  }
  reserveSeat(courseId, studentId) {
    db.query(`UPDATE seats SET open=false, student=$2 WHERE course=$1`, [courseId, studentId]);
  }
}

// 3. Service uses repo — no SQL, easy to test
class EnrollmentService {
  constructor(private repo: EnrollmentRepo) {}

  enroll(courseId: string, studentId: string) {
    const seats = this.repo.findOpenSeats(courseId);
    if (seats.length === 0) throw new Error("No seats");
    this.repo.reserveSeat(courseId, studentId);  // clean domain call
  }
}

// 4. In tests: use a fake repo — no database needed
const fakeRepo: EnrollmentRepo = {
  findOpenSeats: () => [{ id: "s1", open: true }],
  reserveSeat: () => {},
};
const svc = new EnrollmentService(fakeRepo);  // instant, no DB
```

Command pattern — problem

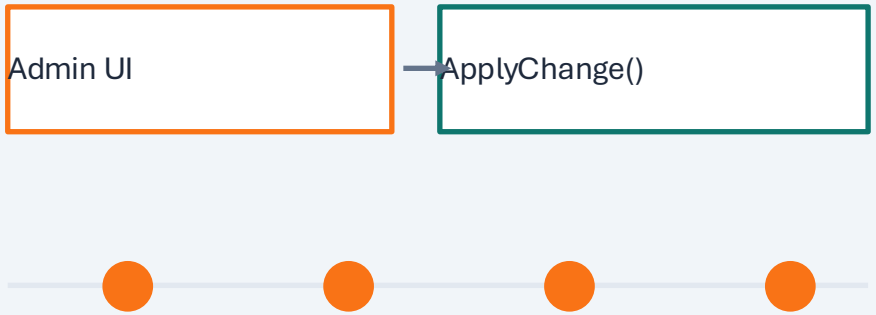
Command

Admins need audit, but direct calls lose intent.

Symptoms

- Coupling the invoker of a request to a particular request should be avoided.
- Auditing means that every single direct call should also implement the audit
- Audit changes mean whole code base should be refactored
- Hard-wired requests should be avoided.
- It should be possible to configure an object (that invokes a request) with a request.
- Direct calls apply changes without a record of intent.

No undo trail



Change history? (missing)

Command pattern — solution

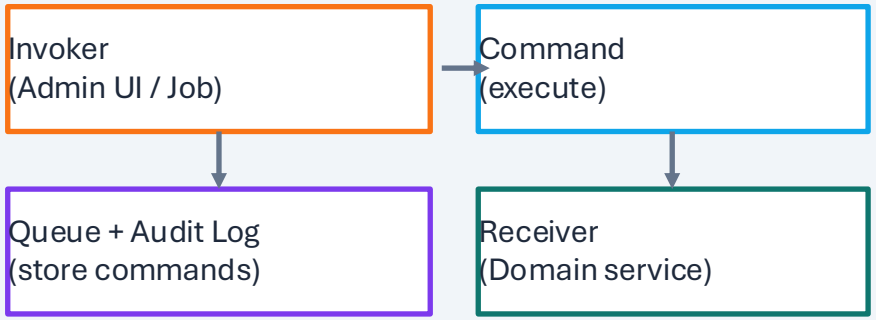
Command

Encapsulate actions as objects with execute() and undo().

How it helps

- Passing the request as input allows for audit and
- Each action is a Command: execute + undo.
- Commands can be queued, logged, and retried safely.
- Audit log stores commands (intent), not just final state.
- Small units make testing easier.

Structure



Tradeoff: more objects. Benefit: traceability + reversibility.

Command pattern — code example

Command

Encapsulate actions as objects with execute() and undo().

```
// 1. Command interface
interface Command {
  execute(): void;
  undo(): void;
  description: string;  // for the audit log
}

// 2. Concrete command — captures intent + supports undo
class DropStudentCmd implements Command {
  description = "Drop student from course";
  constructor(private courseId: string, private studentId: string) {}

  execute() { enrollmentService.drop(this.courseId, this.studentId); }
  undo()    { enrollmentService.enroll(this.courseId, this.studentId); } // reverse!
}

// 3. Invoker stores commands for audit + undo
class AdminInvoker {
  private history: Command[] = [];

  run(cmd: Command) {
    cmd.execute();
    this.history.push(cmd);  // audit trail
    console.log(`[AUDIT]`, cmd.description);
  }
  undoLast() { this.history.pop()?.undo(); }  // undo!
}

// 4. Usage
const admin = new AdminInvoker();
admin.run(new DropStudentCmd("CS101", "stu42"));  // logged + reversible
admin.undoLast();  // mistake? just undo
```

State pattern — problem

State

Workflows evolve over time; scattered transitions create bugs.

Symptoms

- Ride status: requested → matched → en route → completed.
- Conditionals for transitions spread across the codebase.
- Invalid transitions slip in (e.g., complete without match).
- Adding a new state means touching many if/else blocks.

Scattered transitions

```
if (ride.status === 'requested') {
  // ...
} else if (ride.status === 'matched') {
  // ...
} else if (...) {
  // ...
}
```

Same logic duplicated in many places

Hard to ensure “only valid moves”.

State pattern — solution

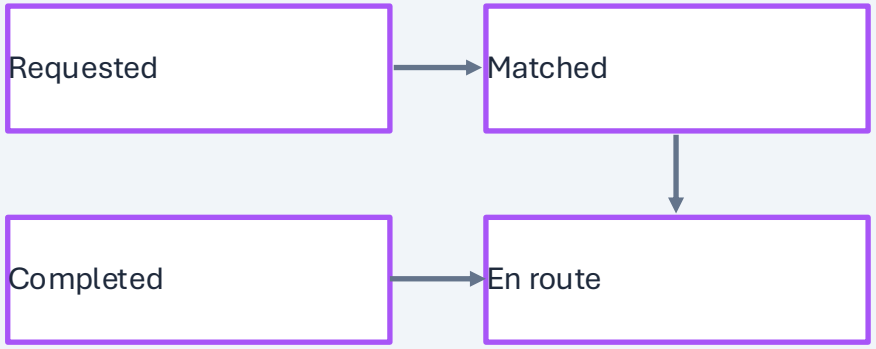
State

Put behavior + transitions inside explicit state objects.

How it helps

- Each state object defines allowed transitions.
- Invalid transitions are blocked by design.
- New states = add a class (localized change).
- Tests focus on transitions (higher confidence).

State machine (example)



Only allowed transitions exist.

State pattern — code example

State

Each state object defines allowed transitions; invalid moves are blocked.

```
// 1. State interface
interface RideState {
  match(driverId: string): RideState;
  startRide(): RideState;
  complete(): RideState;
}

// 2. Concrete states — each defines valid transitions only
class Requested implements RideState {
  match(driverId)  { return new Matched(driverId); }  // valid
  startRide()       { throw new Error("Not matched yet"); }  // blocked!
  complete()        { throw new Error("Not started yet"); }  // blocked!
}
class Matched implements RideState {
  constructor(private driverId: string) {}
  match()         { throw new Error("Already matched"); }
  startRide()     { return new EnRoute(this.driverId); }  // valid
  complete()      { throw new Error("Still en route"); }
}

// 3. Ride context delegates to current state
class Ride {
  private state: RideState = new Requested();
  match(id: string) { this.state = this.state.match(id); }
  start()             { this.state = this.state.startRide(); }
}

// ride.start() before ride.match() → throws "Not matched yet"
```

Strategy vs State — how to choose

Input-based variability vs lifecycle-based variability.

Mohamed bin Zayed University of Artificial Intelligence

Decision cue

Ask: “Why does behavior change?”

- Strategy: choose behavior based on input/context.
- State: behavior changes over time as the object evolves.
- Avoid mixing unless you can clearly name both forces.

Quick comparison

Pattern	Best fit
Strategy	Pricing/eligibility rules, pluggable algorithms
State	Workflows, lifecycles, “valid transitions”

Tip: if you draw a state diagram, you probably want State.

Combining patterns in one feature

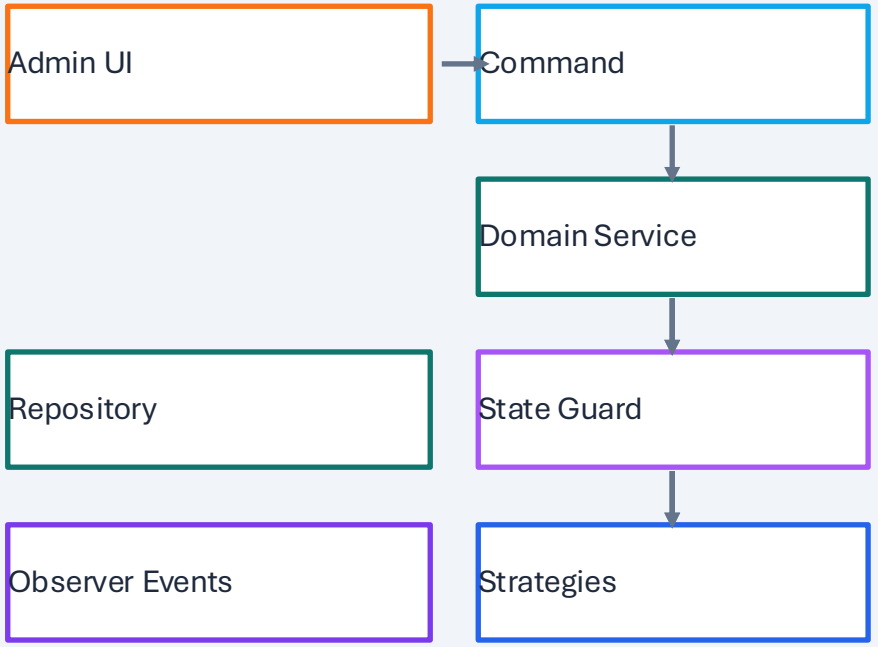
Patterns stack to reduce risk across logic, data, and notifications.

Mohamed bin Zayed University of Artificial Intelligence

Example: enrollment change

- Command captures admin intent and enables undo + audit.
- Repository isolates persistence and makes side effects visible.
- State guards valid phases of an enrollment.
- Observer notifies students and analytics after commit.
- Strategy selects pricing/eligibility rules by context.

Architecture sketch



One feature, multiple seams (each testable on its own).

Adopt patterns safely

Incremental refactors + a simple review checklist.

Mohamed bin Zayed University of Artificial Intelligence

Refactoring paths (incremental)

1) If/Else → Strategy

- Write tests for current outcomes
- Extract one rule into a strategy
- Replace branches step-by-step

2) Direct calls → Observer

- Add an event object
- Emit event without changing behavior
- Move one subscriber at a time + add correlation IDs

3) Inline SQL → Repository

- Inventory queries by domain concept
- Move SQL into repository methods
- Test repos + services separately

Code review checklist + takeaway

- Does this solve a real recurring problem here?
- Is it the smallest pattern that fits?
- Are responsibilities clearer and more testable?
- Are tradeoffs documented (e.g., ADR)?

Lessons learned

- Use fewer patterns well; avoid pattern “style theater”.
- Always ask what cost you are paying for flexibility.
- Patterns are easiest to adopt via safe, incremental refactors.

Document decisions with short ADRs (problem → decision → consequences).